# ComplyDesk — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

## The final layout

```
<repo folder>/
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── compliance_checker.py
│   │   └── drafter.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── prompts/
│   │   ├── compliance.txt
│   │   └── drafter.txt
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py
│   │   └── prompts.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── formatting.py
│   └── pipeline.py
├── tests/
│   ├── integration/
│   │   └── test_live_tool.py
│   └── unit/
│       ├── test_config.py
│       ├── test_formatting.py
│       └── test_pipeline.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
```

```
├── mcp_server.py
└── requirements.txt
```

## Setup files

`requirements.txt`: `mcp[cli]`, `langchain-openai`, `python-dotenv`, `pydantic`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO` / `MAX_REVISION_ROUNDS=3` / `REPLY_TIMEOUT_SECONDS=60`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

## Settings, errors, logging, prompts

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str
    max_revision_rounds: int
    timeout_seconds: float

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    load_dotenv()
    return Config(
```

```
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        max_revision_rounds=int(os.getenv("MAX_REVISION_ROUNDS", "3")),
        timeout_seconds=float(os.getenv("REPLY_TIMEOUT_SECONDS", "60")),
    )
```

```
# src/utils/logger.py
import logging
import os
import sys

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler(sys.stderr)
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/utils/prompts.py

def load_prompt(name: str) -> str:
    with open("src/prompts" + "/" + name + ".txt") as prompt_file:
        return prompt_file.read().strip()
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

## The Drafter

```
# src/prompts/drafter.txt
Write a friendly, concise reply to the customer's support ticket. Be helpful
and apologetic where appropriate.

{feedback_block}

Ticket: {ticket}
```

```
# src/agents/drafter.py
# The Drafter: write a reply. With feedback, fix exactly the named compliance
# issues (not a rewrite).
from dataclasses import dataclass

from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.utils.prompts import load_prompt

@dataclass
class Draft:
    ticket: str
    reply: str

def run_drafter(model, ticket: str, feedback=None) -> Draft:
    if feedback:
        feedback_block = (
            "A compliance check rejected your previous reply for these "
            f"reasons - fix ONLY these, keep the rest:\n{feedback}"
        )
    else:
        feedback_block = ""
    prompt = ChatPromptTemplate.from_template(load_prompt("drafter"))
    chain = prompt | model | StrOutputParser()
    reply = chain.invoke({"ticket": ticket, "feedback_block": feedback_block})
    return Draft(ticket=ticket, reply=reply)
```

## The Compliance-Checker

```
# src/prompts/compliance.txt
You check a draft support reply against company policy. Approve ONLY if it is
compliant. Reject (with a specific issue for each problem) if it:
- promises a full refund (agents may offer up to a 10% credit; full refunds
  require manager approval and must be offered as an escalation, not a promise),
```

```
- uses an absolute guarantee ("I guarantee", "I promise", "100%"),
- leaks internal information (internal tools, other customers, costs).

Ticket: {ticket}
Draft reply:
{reply}
```

```
# src/agents/compliance_checker.py
# The Compliance-Checker: a checked verdict with specific issues. Real policy
# criteria in the prompt, so it doesn't approve everything on round 1.
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel

from src.utils.prompts import load_prompt

class ComplianceVerdict(BaseModel):
    approved: bool
    issues: list

def run_compliance(model, draft) -> ComplianceVerdict:
    prompt = ChatPromptTemplate.from_template(load_prompt("compliance"))
    chain = prompt | model.with_structured_output(ComplianceVerdict)
    return chain.invoke({"ticket": draft.ticket, "reply": draft.reply})
```

## The pipeline

```
# src/pipeline.py
# run_pipeline: Drafter writes -> Compliance-Checker checks -> approved? stop;
# else feed the issues back and go again, up to max_rounds. The Progress
# holder records each round (for the trace) and the latest draft (for the
# partial fallback on timeout).
from dataclasses import dataclass

from src.agents.compliance_checker import run_compliance
from src.agents.drafter import run_drafter
from src.utils.logger import get_logger

@dataclass
class PipelineResult:
    ticket: str
    final_reply: str
    approved: bool
    rounds: list
```

```
class Progress:
    def __init__(self):
        self.latest_draft = None
        self.rounds = []

def run_pipeline(model, ticket, max_rounds, progress=None) -> PipelineResult:
    logger = get_logger(__name__)
    if progress is None:
        progress = Progress()
    draft = run_drafter(model, ticket)
    for round_number in range(1, max_rounds + 1):
        progress.latest_draft = draft.reply
        verdict = run_compliance(model, draft)
        progress.rounds.append({
            "round": round_number,
            "approved": verdict.approved,
            "issues": verdict.issues,
        })
        logger.info(f"round {round_number}: approved={verdict.approved}")
        if verdict.approved:
            return PipelineResult(ticket, draft.reply, True, progress.rounds)
        feedback = "\n".join(f"- {issue}" for issue in verdict.issues)
        draft = run_drafter(model, ticket, feedback)
    progress.latest_draft = draft.reply
    return PipelineResult(ticket, draft.reply, False, progress.rounds)
```

## The formatter

```
# src/formatting.py
# Turn pipeline output into the string the MCP tool returns. Pure functions.
def format_result(result) -> str:
    lines = [f"Reply for ticket: {result.ticket}", "", "Compliance trace:"]
    for entry in result.rounds:
        state = "approved" if entry["approved"] else "rejected"
        lines.append(f"  Round {entry['round']}: {state}")
        for issue in entry["issues"]:
            lines.append(f"    - {issue}")
    if result.approved:
        status = f"approved after {len(result.rounds)} round(s)."
    else:
        status = (f"NOT approved; stopped at the "
                  f"{len(result.rounds)}-round cap - send to a human.")
    lines.append("")
    lines.append(f"Status: {status}")
    lines.append("")
```

```
    lines.append("Reply:")
    lines.append(result.final_reply)
    return "\n".join(lines)

def format_partial(ticket, progress) -> str:
    draft = progress.latest_draft or "(no draft produced yet)"
    return (
        f"Reply for ticket: {ticket}\n\n"
        f"Status: PARTIAL - timed out; returning the best draft so far "
        f"({len(progress.rounds)} round(s)). Review before sending.\n\n"
        f"Draft so far:\n{draft}"
    )
```

## The MCP tool

```
# mcp_server.py
# Exposes the whole Drafter/Compliance-Checker pipeline as ONE MCP tool. The
# pipeline is synchronous, so it runs on a worker thread (asyncio.to_thread)
# wrapped in asyncio.wait_for, so the timeout actually fires. On timeout, the
# Progress holder gives the best draft so far. Logging -> stderr only.
# Run it with:  mcp dev mcp_server.py
import asyncio
import sys

from mcp.server.mcpserver import MCPServer

from src.config import load_config
from src.formatting import format_partial, format_result
from src.models.llm import get_model
from src.pipeline import Progress, run_pipeline
from src.utils.logger import get_logger

CONFIG = load_config()
MODEL = get_model(CONFIG)
logger = get_logger(__name__)

mcp = MCPServer("complydesk-service")

@mcp.tool()
async def draft_safe_reply(ticket: str) -> str:
    """Draft a reply to a customer ticket that has been checked against
    company policy (refunds, guarantees, information leaks). Returns the reply
    plus a compliance trace, or a marked partial if it runs past the timeout."""
    logger.info(f"draft_safe_reply: {ticket[:50]}")
    progress = Progress()
```

```
    try:
        result = await asyncio.wait_for(
            asyncio.to_thread(
                run_pipeline, MODEL, ticket, CONFIG.max_revision_rounds,
                progress,
            ),
            timeout=CONFIG.timeout_seconds,
        )
        return format_result(result)
    except asyncio.TimeoutError:
        logger.warning("draft_safe_reply timed out")
        return format_partial(ticket, progress)

if __name__ == "__main__":
    print("complydesk service starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

> **Two things that matter:** (1) `asyncio.to_thread` runs the *synchronous* `run_pipeline` on a worker thread so the event loop stays free and `wait_for` can fire — wrapping a blocking call directly in `wait_for` would just block. (2) `to_thread` can't cancel the worker thread, so the pipeline keeps running in the background after a timeout — a known limitation; a real deployment adds a cancel flag the loop checks each round.

## main.py (standalone)

```
# main.py
import sys

from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import get_model
from src.pipeline import run_pipeline

TICKETS = [
    "My order #1001 arrived late - I want a full refund.",
    "How do I change the email on my account?",
]

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
```

```
        sys.exit(1)
    model = get_model(config)
    for ticket in TICKETS:
        result = run_pipeline(model, ticket, config.max_revision_rounds)
        print(f"\n=== {ticket} ===")
        for entry in result.rounds:
            mark = "approved" if entry["approved"] else "rejected"
            print(f"  Round {entry['round']}: {mark}")
        print(f"approved={result.approved}\n{result.final_reply}")

if __name__ == "__main__":
    main()
```

## The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "COMPLYDESK_TEST_SETTING"

def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting() -> None:
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
```

```
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    if config.max_revision_rounds <= 0:
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_formatting.py
# The formatter, on hand-made results — no model.
# Run from the repo root: python -m tests.unit.test_formatting
from src.formatting import format_partial, format_result

class FakeResult:
    def __init__(self, ticket, final_reply, approved, rounds):
        self.ticket = ticket
        self.final_reply = final_reply
        self.approved = approved
        self.rounds = rounds

class FakeProgress:
    def __init__(self, latest_draft, rounds):
        self.latest_draft = latest_draft
        self.rounds = rounds

def test_result_shows_trace() -> None:
    result = FakeResult(
        "refund", "We can offer a 10% credit.", True,
        [{"round": 1, "approved": False, "issues": ["promised full refund"]},
         {"round": 2, "approved": True, "issues": []}],
    )
    out = format_result(result)
    ok = ("Round 1: rejected" in out and "- promised full refund" in out
          and "Round 2: approved" in out
          and "approved after 2 round(s)" in out
          and "We can offer a 10% credit." in out)
    print(f"case 1 (trace + status + reply)  -> {'OK' if ok else 'WRONG'}")

def test_cap_status_says_human() -> None:
    result = FakeResult(
        "x", "draft", False,
        [{"round": 1, "approved": False, "issues": ["x"]},
```

```
         {"round": 2, "approved": False, "issues": ["x"]}],
    )
    out = format_result(result)
    ok = "send to a human" in out
    print(f"case 2 (cap -> send to human)    -> {'OK' if ok else 'WRONG'}")

def test_partial() -> None:
    out = format_partial("slow", FakeProgress("half a reply", [{"round": 1}]))
    ok = "PARTIAL" in out and "half a reply" in out
    print(f"case 3 (partial names the draft) -> {'OK' if ok else 'WRONG'}")

if __name__ == "__main__":
    test_result_shows_trace()
    test_cap_status_says_human()
    test_partial()
```

```
# tests/unit/test_pipeline.py
# The revise loop and the round cap, with a tiny FAKE model (no key).
# Run from the repo root: python -m tests.unit.test_pipeline
from langchain_core.messages import AIMessage

from src.agents.compliance_checker import ComplianceVerdict
from src.pipeline import run_pipeline

class FakeModel:
    def __call__(self, prompt_value):
        text = prompt_value.to_string()
        if "compliance check rejected" in text:
            return AIMessage(content="We can offer a 10% credit or escalate.")
        return AIMessage(content="Sure, here is your full refund.")

    def with_structured_output(self, schema):
        def check(prompt_value):
            text = prompt_value.to_string()
            if "full refund" in text.lower():
                return ComplianceVerdict(
                    approved=False,
                    issues=["Promised a full refund; offer a 10% credit."],
                )
            return ComplianceVerdict(approved=True, issues=[])
        return check

class AlwaysReject(FakeModel):
    def with_structured_output(self, schema):
        def check(prompt_value):
            return ComplianceVerdict(approved=False, issues=["nitpick"])
```

```
        return check

def test_rejects_then_approves() -> None:
    result = run_pipeline(FakeModel(), "refund please", max_rounds=3)
    ok = result.approved and len(result.rounds) == 2
    if "credit" not in result.final_reply.lower():
        ok = False
    print(f"case 1 (refund fixed, approved r2) -> {'OK' if ok else 'WRONG'}")

def test_stops_at_cap() -> None:
    result = run_pipeline(AlwaysReject(), "anything", max_rounds=2)
    ok = (not result.approved) and len(result.rounds) == 2
    print(f"case 2 (never approved -> cap)     -> {'OK' if ok else 'WRONG'}")

if __name__ == "__main__":
    test_rejects_then_approves()
    test_stops_at_cap()
```

```
# tests/integration/test_live_tool.py
# Launches the real ComplyDesk MCP server and calls its tool. Run from the
# repo root: python -m tests.integration.test_live_tool
import asyncio
import os

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def call_tool(ticket, env=None):
    params = StdioServerParameters(
        command="python", args=["mcp_server.py"], env=env
    )
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            result = await session.call_tool(
                "draft_safe_reply", {"ticket": ticket}
            )
            return result.content[0].text

async def test_returns_trace_and_reply() -> None:
    text = await call_tool("My order #1001 was late - I want a full refund.")
    ok = "Compliance trace" in text and "Reply:" in text
    print(f"case 1 (trace + reply)   -> {'OK' if ok else 'WRONG'}")

async def test_timeout_gives_partial() -> None:
    env = dict(os.environ)
```

```
    env["REPLY_TIMEOUT_SECONDS"] = "0.001"
    text = await call_tool("anything", env=env)
    ok = "PARTIAL" in text
    print(f"case 2 (timeout -> PARTIAL) -> {'OK' if ok else 'WRONG'}")

async def main() -> None:
    await test_returns_trace_and_reply()
    await test_timeout_gives_partial()

if __name__ == "__main__":
    asyncio.run(main())
```

**Expected unit output** (all `OK`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (trace + status + reply)  -> OK
case 2 (cap -> send to human)    -> OK
case 3 (partial names the draft) -> OK
```

```
case 1 (refund fixed, approved r2) -> OK
case 2 (never approved -> cap)     -> OK
```

**Expected integration output** (real model):

```
case 1 (trace + reply)   -> OK
case 2 (timeout -> PARTIAL) -> OK
```

That's the whole service: a Drafter/Compliance-Checker loop, capped, exposed as one `draft_safe_reply` MCP tool, reporting its compliance trace, and always returning something — a policy-safe reply, or a marked partial — instead of hanging.